



Correction de la Série d'Exercices TD/TP 02

Module : Algorithmique et Python 2

Filière : MIP

Semestre : S2

Etablissement : FSSM

Année Universitaire : 2023/2024

Corrigé par : Passerelle, Centre de Formation et d'Orientation

Adresse : LOT SIDI ABBAD 1 IMM N 298 ETG 3 BUR N 16

Email : info@passerelle.ma

Téléphone : +212 666 92 44 62 / +212 524 30 73 09

Introduction

Chers étudiants,

Ce document présente la correction de la série d'exercices TD/TP 2 du module "Algorithmique et Python 2" de la filière MIP, semestre S2, pour l'année universitaire 2023/2024, à la Faculté des Sciences Semlalia de Marrakech (FSSM).

L'objectif de cette série d'exercices est de vous familiariser avec les fonctions en Python, la manipulation des listes, et l'approfondissement de votre compréhension des structures de contrôle itératives et conditionnelles. Vous aborderez également des notions essentielles telles que la distinction entre les fonctions et les procédures à travers des exemples spécifiques qui renforceront vos compétences en programmation et vous aideront à comprendre les mécanismes internes du langage Python.

Les corrections fournies dans ce document comprennent des explications détaillées pour chaque question, illustrées par des exemples de code et des descriptions des démarches nécessaires pour résoudre les problèmes proposés. Cela vous permettra de mieux saisir les concepts abordés et de développer votre capacité à appliquer ces connaissances dans des contextes pratiques.

Nous espérons que ce document vous sera utile dans votre apprentissage et qu'il vous aidera à maîtriser les sujets traités. N'hésitez pas à contacter votre professeur pour toute question ou clarification supplémentaire.

Bonne lecture et bon travail !

Professeur : ER-RAHMOUNY Hamza.

Exercice 01 :

1. Pourquoi utilise-t-on les fonctions en programmation ?

Les fonctions sont utilisées en programmation pour éviter la redondance du code lors de l'exécution de tâches répétitives et similaires. Elles permettent de regrouper un ensemble d'instructions sous un nom unique, facilitant ainsi la réutilisation, la maintenance et la clarté du code.

2. Expliquez la différence entre une fonction et une procédure en programmation.

Donnez des exemples pour illustrer chacun des concepts.

La différence principale entre une fonction et une procédure réside dans leur retour de valeur :

_ Fonction : Une fonction est un bloc de code réutilisable qui exécute une tâche spécifique et retourne une valeur. Par exemple :

```
def addition(a, b):  
    return a + b
```

_ Procédure : Une procédure est un bloc de code réutilisable qui exécute une tâche spécifique sans retourner de valeur. Par exemple :

```
def afficher_message():  
    print("Bonjour, tout le monde!")
```

Exercice 02 :

```
def afficher_resultats():
```

```
    a = 10
```

```
    b = 5
```

```
    print(a + b)
```

```
afficher_resultats() # Qu'affiche cet appel ?
```

- Cette appel de la procédure afficher_resultats(), va afficher la valeur 15.

```
def afficher_resultats():
```

```
    a = 10
```

```
    b = 5
```

```
    return a + b
```

```
resultat = afficher_resultats()
```

```
print(resultat) # Qu'affiche cet appel ?
```

- Cette appel de la fonction afficher_resultats(), va afficher la valeur 15.

```
c = 7
def afficher_resultats():
    a = 10
    b = 5
    print(a + b + c)
afficher_resultats() # Qu'affiche cet appel ?
```

- Cette appel de la procédure afficher_resultats(), va afficher la valeur 22.

```
Def afficher_resultats(a, b, c):
    print(a + b + c)
afficher_resultats(a=1, 2, 3) # Qu'affiche cet appel ?
```

- Erreur, car par définitions, les arguments par mots clés doivent être après les arguments positionnels.

Exercice 03 :

1. Fonction de multiplication :

Écrire une fonction appelée « multiplication » qui prend deux paramètres et renvoie leur produit.

Proposition 1 de réponse :

```
def multiplication(a, b) :
    return a * b          #En utilise le symbole * pour la multiplication et non pas le symbole x.
```

Proposition 2 de réponse :

```
def multiplication(a, b) :
    p = a * b
    return p
```

2. Fonction de vérification de palindrome :

Écrire une fonction appelée « palind » qui prend une chaîne de caractères en entrée et renvoie True si la chaîne est un palindrome, sinon False.

Exemple de palindrome : radar)

Proposition 1 de réponse :

```
def palind(s) :
    if ( s[ : ] == s[ : :-1] ) :          #Comparer la chaine de caractères avec son inverse.
        return True                      #Si la chaine et son inverse sont similaire donc sortir avec True.
    return False                         #Si la fonction ne return pas True donc c'est obligatoirement False.
```

Proposition 2 de réponse :

```
def palind(s) :  
    sTemp = list(s)          #Convertir la chaine s en liste avec la fonction list().  
    if (sTemp == sTemp.reverse()) :          #Comparer la liste avec son inverse.  
        return True          #Si la liste et son inverse sont similaire donc sortir avec True.  
    return False             #Si la fonction ne return pas True donc c'est obligatoirement False.
```

Proposition 3 de réponse :

```
Def palind(s) :  
    n = len(s)               # n est le nombre d'éléments de s.  
    for i in range(n) :      # i va prendre les valeur suivantes (0, 1, ..., n-1)  
        if ( s[i] != s[n-i-1] ) :      # Comparant un élément de s avec son opposé de l'autre sens  
            return False              #Si vrai donc ce n'est pas palindrome.  
    return True              # Si la boucle se termine sans False donc chaque élément similaire à son  
                             # opposé dans l'autre sens donc c'est obligatoirement palindrome
```

3. Fonction de la somme cumulative :

Écrire une fonction appelée « cumul » qui prend une liste de nombres en entrée et renvoie une nouvelle liste où chaque élément est la somme cumulative des éléments précédents jusqu'à cet indice.

Exemple : [1, 2, 3, 4] ----- >> [1, 3, 6, 10]

```
def cumul(a) :  
    s = 0                #Initialiser la variable s par 0 pour l'utiliser dans la somme cumulative.  
    aCumul = []          #Initialiser une liste vide, pour l'utiliser pour stocker les sommes cumulatives.  
    for x in a :          # x va prendre dans chaque itération une valeur de la liste a.  
        s = s + x        # s va rajouter à chaque fois x dans s, donc vers la fin s = 0+1+2+3+4  
        aCumul.append(s) # dans chaque itération on ajoute dans aCumul la valeur de s.  
    return aCumul
```

4. Fonction de la table de multiplication

Écrire une fonction appelée « TM » qui prend un nombre entre 1 et 9 et affiche la table de multiplication de ce nombre.

Exemple : TM(4) → 4*1=1, 4*2=8, 4*3=12,, 4*10=40

```

def    TM(n) :
    if n < 1 or n > 9 :      #On traite que les tables de multiplication des nombres entre 1 et 9.
        print("Le nombre doit : 1<= n <= 9")
    else :                  #Si le nombre compris entre 1 et 9, on affiche sa table de multiplication.
        for i in range(1, 11) :      # i va prendre ces valeurs (1,2,3,4,5,6,7,8,9,10).
            m = n * i
            print(n, " * ", i, " = ", m)

```

5. Fonction des nombres premiers

Écrire une fonction appelée « nombre_premier » qui prend un nombre entier et renvoie True si le nombre est premier, sinon renvoie False.

```

def    nombre_premier(n) :
    if ( n <= 1 ) :          #Par définition les nombres <=1 ne sont pas premiers.
        return False
    else :
        for i in range(2,n) :      # i va prendre ces valeurs (2,3, ..., n-1).
            if (n%i == 0) :        #Si n est divisible par i, donc n n'est pas premier.
                return False
        return True             #Si aucun i ne divise pas n vers la fin de la boucle, donc n est premier.

```

6. Fonction calcule de la médiane des nombres dans une liste

Écrire une fonction appelée « médiane » qui prend une liste de nombres et renvoie la valeur médiane.

```

def    mediane(T) :
    T.sort()                  #Le calcul du médiane se fait sur des tableaux triés.
    n = len(T)                # n est le nombre des éléments de la liste T.
    if(n%2 == 0) :           #Est ce que le nombre des éléments est pair ?
        return (T[n//2]+T[n//2-1]) / 2      #Si pair, c'est la définition de la médiane.
    else :
        return T[n//2]       #Si impair, c'est la définition de la médiane.

```

7. Fonction dessiner_triangle

Écrire une fonction appelée « desTri » qui prend la hauteur, et dessine un triangle en utilisant que les print('*').

```

def    destri(h) :
    for i in range(1, h+1) :      # i va prendre ces valeurs : (1, 2, ..., h).
        print(i * " * ")        #L'affichage de nombre i étoile dans la même ligne.

```

8. Fonction dessiner_cercle

Écrire une fonction appelée `desCer` qui prend le rayon, et dessine un cercle en utilisant que les `print('*')`.

Il n'y a pas de solution disponible pour cet exercice en utilisant uniquement les connaissances et les compétences actuellement acquises par les étudiants de S2 MIP.

Exercice 04 : Calcul de la Moyenne Pondérée et gestion des variables globales et locales.

Écrivez un programme en Python qui calcule la moyenne pondérée de différents examens. Le programme doit utiliser une variable globale pour stocker le coefficient total et une variable globale pour stocker la somme des produits des notes par les coefficients.

1. Initialisez une variable globale « `coefficient_total` » avec la valeur 0.
2. Écrivez une fonction « `ajouter_note` » qui prend en paramètre une note et un coefficient, ajoute le produit de la note par le coefficient à la variable globale des sommes des produits par les coefficients, et ajoute le coefficient à la variable globale « `coefficient_total` ».
3. Écrivez une fonction « `calculer_moyenne` » qui calcule la moyenne pondérée en divisant la somme des produits des notes par les coefficients par le « `coefficient_total` ».
4. Testez votre programme en ajoutant quelques notes avec des coefficients différents.

```
coefficient_total = 0          #Initialisation par 0
sommeNC = 0                   #La variable qui va cumuler la somme des produits des notes par les coefficients
def ajouter_note(n, c) :
    global coefficient_total, sommeNC    #les deux variables utilisent leurs valeurs globales.
    coefficient_total += c              #Ajouter le nouveau coefficient au cumule des coefficients.
    sommeNC += n * c                   #Ajouter le produit n * c au cumule de la somme des produits.
def calculer_moyenne() :
    return sommeNC / coefficient_total

#Tester le programme :
ajouter_note(17, 5)  #Ajouter la note 17 avec le coefficient 5
ajouter_note(13, 2)  #Ajouter la note 13 avec le coefficient 2
ajouter_note(15, 1)  #Ajouter la note 15 avec le coefficient 1
resultat = calculer_moyenne()          #Calculer cette moyenne (17*5+13*2+15*1)/8
print("La moyenne générale est : ", resultat)    #Affichage du résultat
```